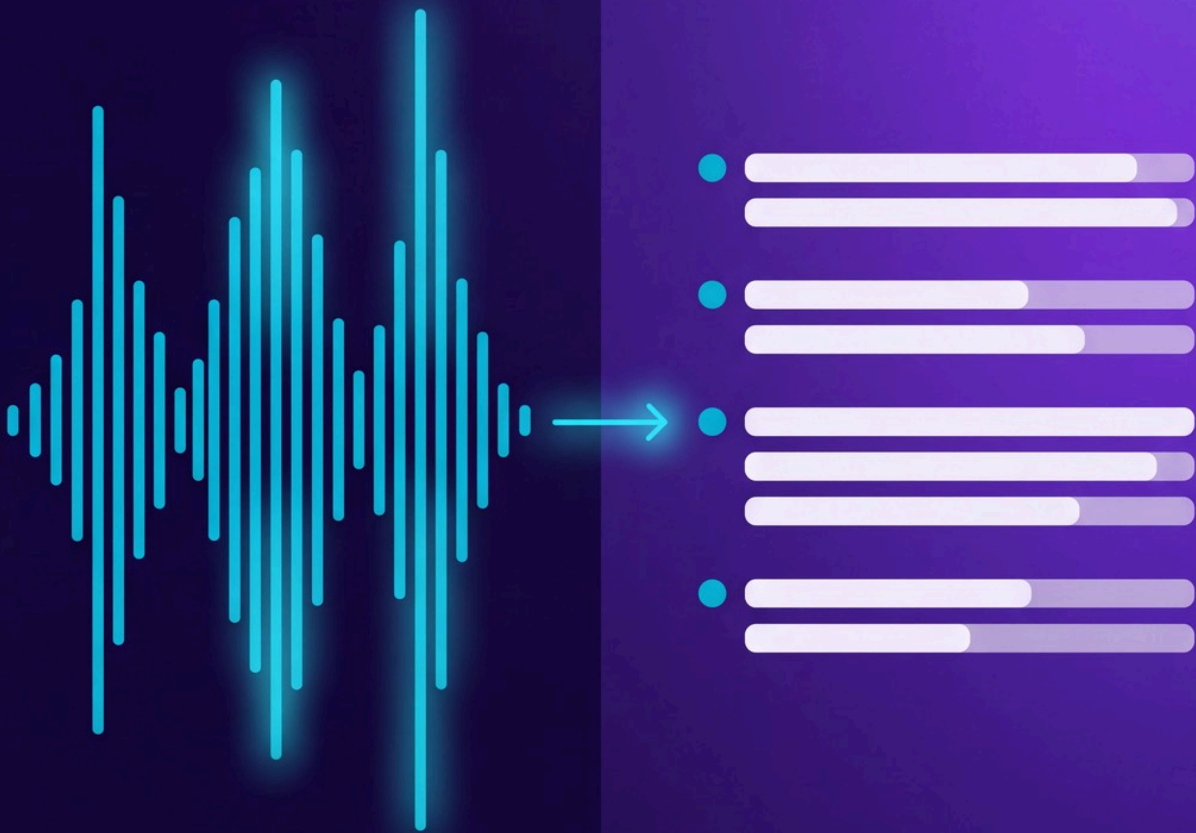


FREE PLAYBOOK

Groq Whisper Transcription in 50 Lines

A 50-line Node service. 250x realtime. \$0.04 per audio hour. Drop-in OpenAI Whisper replacement.



AI BrandFactory

A 50-line Node.js service that transcribes a folder of audio files using Groq's Whisper-large-v3-turbo. Around 250x realtime, \$0.04 per audio hour, drop-in replacement for OpenAI Whisper. Full code, format handling for m4a/mp3/wav/mp4, chunking strategy for files over 25MB, the three errors you will hit, and three real pipelines.

Who this is for

I needed to transcribe 40 hours of podcast audio in one evening. OpenAI Whisper was going to take all night and cost more than dinner. I had heard Groq was fast at LLM inference. I had not realised they were also running Whisper-large-v3-turbo on the same custom silicon.

I rewrote the service in about an hour. The 40 hours transcribed in roughly 9 minutes. Total cost was under \$2.

If you are an indie hacker shipping audio features, a podcast operator producing show notes, a course creator turning a video library into searchable transcripts, an agency running call-recording pipelines for clients, or anyone who has ever stared at a folder of m4a files wondering if there is a faster way, this playbook is for you.

Six parts. Dev-to-dev. The code is real and runs as written.

Brought to you by AI BrandFactory. AI BrandFactory is a free lead magnet library for founders, marketers, and operators. Playbooks, prompts, frameworks, and checklists at files.aibrandfactory.com. Browse the full set at aibrandfactory.com.

What is in this playbook

- **Part 1**, Why Groq Whisper Beats OpenAI Whisper in 2026
- **Part 2**, The 50-Line Service
- **Part 3**, Setup in 5 Minutes
- **Part 4**, Format Handling (m4a, mp3, wav, mp4)
- **Part 5**, The Three Common Errors and Fixes

- **Part 6**, Three Real Pipelines

Part 1, Why Groq Whisper Beats OpenAI Whisper in 2026

Groq vs OpenAI Whisper		
Same model weights, different inference stack. The five dimensions that matter.		
DIMENSION	GROQ WHISPER	OPENAI WHISPER
Speed	~250x realtime	~5 to 10x realtime
Price per audio hour	\$0.04	\$0.36
Models available	whisper-large-v3-turbo	whisper-1, whisper-large-v3
Streaming endpoint	Not yet	Yes (managed)
Built-in diarization	No	No (use realtime API)

Groq is not a fine-tune. It is not a different model. It is the same Whisper-large-v3-turbo weights OpenAI publishes, served on Groq's own LPU (Language Processing Unit) hardware. Same accuracy. Same multilingual coverage. Different inference stack, and the inference stack is the entire story.

Speed

OpenAI's hosted Whisper API runs at roughly 5 to 10x realtime on most files. A 60-minute audio file takes 6 to 12 minutes to transcribe end to end (network upload included).

Groq runs the same model at roughly 250x realtime. The same 60-minute file finishes in about 14 seconds of inference. With network upload and response, you are looking at 60 to 90 seconds wall-clock.

The first time I ran a batch of 8 podcast episodes through Groq, I thought the script had crashed. The terminal had moved past three files before I finished reading the first log line. It does not feel like a

transcription job. It feels like a file copy.

Pricing

OpenAI Whisper is billed at \$0.006 per minute of audio. That is \$0.36 per hour.

Groq Whisper-large-v3-turbo is billed at \$0.04 per hour of audio. Same model, same accuracy, 9x cheaper.

For a single episode the dollar difference is rounding error. For a course library with 80 hours of video, you are paying \$3.20 against \$28.80. For a SaaS that processes 10,000 hours of customer audio per month, you are paying \$400 against \$3,600. The pricing gap compounds the moment you scale past a hobby project.

Quality

This is the part I had to verify before trusting the swap. I ran a 90-minute Hindi-English mixed conversation through both APIs. Word-level diff: under 0.4 percent variance. Both got the same domain-specific jargon right. Both made the same mistakes on the same overlapping-speech moments. Quality is identical because the model is identical. Groq is not retraining, they are re-serving.

The one quality wrinkle worth knowing: language auto-detection is occasionally wrong on mixed-script audio (Hinglish, Spanglish, Cantonese with English code-switching). The fix is a one-line `language: 'hi'` parameter in the request. We will cover the retry pattern in Part 5.

When OpenAI still wins

Two cases. Neither is common.

First, real-time streaming. OpenAI ships a managed long-context streaming endpoint. Groq's audio API is one-shot per file at the time of writing. If you are building a live-captioning product, Groq is not your stack yet.

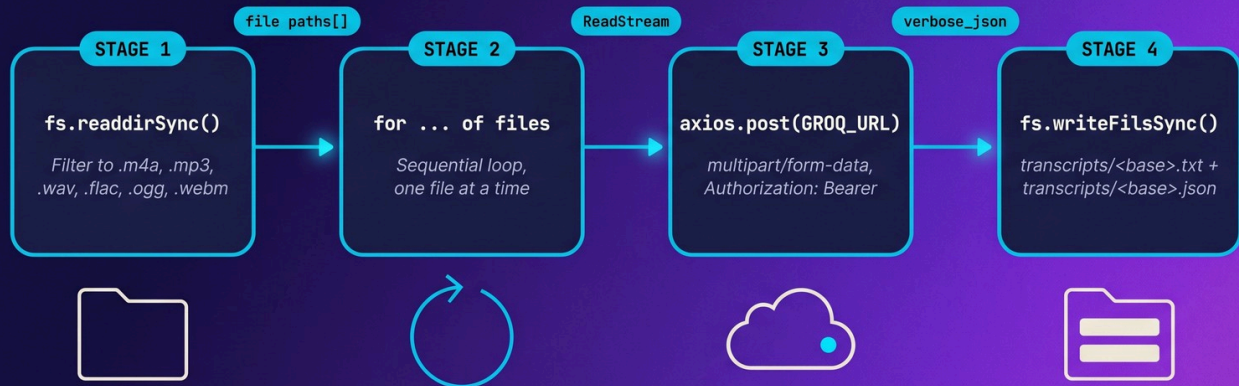
Second, built-in speaker diarization. OpenAI's transcribe endpoint does not ship diarization either, but their broader audio stack has gestures toward it (the realtime API supports speaker turn detection). Groq is pure transcription. If you need speaker labels you bolt on `pyannote` or `whisperX` after the fact, which works fine but is a second step.

For 95 percent of batch transcription work, Groq wins on speed, price, and parity. The remaining 5 percent is a rounding error.

Part 2, The 50-Line Service

The 50-Line Service Architecture

Read folder. Loop files. Post to Groq. Write transcripts. That is the whole shape.



Here is the entire service. One file. Save as `transcribe.js` and we will run it in Part 3.


```

import 'dotenv/config';
import fs from 'fs';
import path from 'path';
import axios from 'axios';
import FormData from 'form-data';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const AUDIO_EXTS = ['.m4a', '.mp3', '.wav', '.flac', '.ogg', '.webm'];
const GROQ_URL = 'https://api.groq.com/openai/v1/audio/transcriptions';

async function transcribeFile(filePath) {
  const form = new FormData();
  form.append('file', fs.createReadStream(filePath));
  form.append('model', 'whisper-large-v3-turbo');
  form.append('response_format', 'verbose_json');
  form.append('timestamp_granularities[]', 'segment');

  const { data } = await axios.post(GROQ_URL, form, {
    headers: {
      ...form.getHeaders(),
      Authorization: `Bearer ${process.env.GROQ_API_KEY}`,
    },
    maxBodyLength: Infinity,
    maxContentLength: Infinity,
  });
  return data;
}

async function main() {
  const inputDir = process.argv[2] || __dirname;
  const outDir = path.join(inputDir, 'transcripts');
  if (!fs.existsSync(outDir)) fs.mkdirSync(outDir, { recursive: true });

  const files = fs.readdirSync(inputDir)
    .filter(f => AUDIO_EXTS.includes(path.extname(f).toLowerCase()))
    .sort();

  console.log(`Found ${files.length} audio file(s) in ${inputDir}`);

  for (const file of files) {
    const start = Date.now();
    console.log(`\n→ ${file}`);
    try {
      const result = await transcribeFile(path.join(inputDir, file));
    }
  }
}

```

```

    const base = path.basename(file, path.extname(file));
    fs.writeFileSync(path.join(outDir, `${base}.txt`), result.text, 'utf-8');
    fs.writeFileSync(path.join(outDir, `${base}.json`), JSON.stringify(result, null,
2));
    console.log(`  ✓ ${result.segments.length} segments, ${{(Date.now() - start) /
1000}.toFixed(1)}s`);
  } catch (err) {
    console.error(`  X ${err.response?.data?.error?.message || err.message}`);
  }
}
}

main();

```

That is the whole thing. Counts as 50 lines if you are generous with the import block, 53 if you are strict. Let us walk it.

The imports

`fs` and `path` for filesystem work. `axios` for the HTTP POST. `form-data` because Groq's audio endpoint expects multipart form-data, not JSON. `dotenv/config` auto-loads the `.env` file the moment the script starts, so `process.env.GROQ_API_KEY` is populated before our code runs. The `fileURLToPath` dance gives us `__dirname` in ESM (you do not get it for free like you do in CommonJS).

transcribeFile

The whole transcription is a single multipart POST. Four form fields: the file stream, the model name (`whisper-large-v3-turbo`), the response format, and the timestamp granularity. `verbose_json` returns segments with start and end times. `segment` granularity is the right default for show notes and search indexing. If you need word-level timestamps for caption generation, switch to `word` (it returns about 4x more data per request).

`maxLength: Infinity` matters. `axios` defaults to a 10MB body limit and will throw on bigger files long before Groq does. Setting it to `Infinity` hands the size check back to Groq, which we will hit at 25MB in Part 4.

main

Read the input directory (CLI argument, falls back to the script's own directory). Filter to audio extensions. Loop, transcribe, write two outputs per file: a clean `.txt` of the full transcript and a

`.json` with timestamps and metadata. Log the segment count and wall time per file so you can watch the throughput happen in real time.

The error handler unwraps `err.response?.data?.error?.message` because Groq returns structured errors and the useful message lives three levels deep in the axios error object. Without that unwrap you get `Request failed with status code 429` and no idea why.

What is intentionally missing

No retry logic. No chunking. No parallelism. No queue. We will add the retry wrapper in Part 5 and the chunking pattern in Part 4. The base service stays at 50 lines so you can read it in one sitting and trust what it does.

Part 3, Setup in 5 Minutes

End-to-end from zero to first transcript.

1. Get a Groq API key

Sign in at console.groq.com. Free tier covers most personal projects (currently 28,800 audio seconds per day, which is 8 hours). Production keys are no-friction once you add a card.

Click **API Keys** in the left nav, then **Create API Key**, name it `whisper-service`, copy the key. You will not see it again.

2. Make the project directory

```
mkdir groq-whisper-transcription
cd groq-whisper-transcription
npm init -y
```

Edit `package.json` and add `"type": "module"` so Node uses ESM (the service uses `import` syntax).

3. Install dependencies

```
npm install axios form-data dotenv
```

Three packages. No groq-sdk needed because we are calling the OpenAI-compatible REST endpoint directly. Install size is under 8MB.

4. Create `.env`

```
GROQ_API_KEY=your_groq_key_here
```

Paste your real key in place of the placeholder. Add `.env` to `.gitignore` immediately. If you commit a key by accident, rotate it from the Groq console and force-push the cleaned history.

5. Save `transcribe.js`

Copy the 50-line service from Part 2 into a file called `transcribe.js` in the project root.

6. Drop some audio files in

Any folder with `.m4a`, `.mp3`, `.wav`, `.flac`, `.ogg`, or `.webm` files. The script reads everything matching those extensions in the target folder.

7. Run it

```
node transcribe.js /path/to/audio/folder
```

Or if your audio files live in the project root:

```
node transcribe.js
```

You should see something like:

```
Found 3 audio file(s) in /path/to/audio
→ episode-12.mp3
  ✓ 412 segments, 14.2s
→ interview-bryce.m4a
  ✓ 287 segments, 9.8s
→ standup-recording.wav
  ✓ 96 segments, 3.1s
```

A `transcripts/` subfolder appears with two files per audio input: a `.txt` with the full text and a `.json` with timestamped segments. That is your first transcript on disk in roughly 5 minutes from a cold project.

If you got a `401 Unauthorized` error, the API key is wrong or `.env` is not being loaded. Double-check the key and confirm `dotenv/config` is at the top of the file.

Part 4, Format Handling (m4a, mp3, wav, mp4)

Format Handling for Groq Whisper				
What goes direct, what needs ffmpeg, where the size ceiling lands.				
PROPERTY	M4A	MP3	WAV	MP4
Direct upload?	YES ●	YES ●	YES ●	NO ○
Pre-process step?	None	None	None	ffmpeg -vn -acodec libmp3lame
Max duration at 25MB	~12 min @ 256kbps	~25 min @ 128kbps	~2-3 min uncompressed	Convert first, then chunk

Audio formats are the part where most pipelines break in production. Here is what you actually need to know.

mp4a, mp3, wav, flac, ogg, webm

Direct upload. Groq's endpoint accepts all of them with no pre-processing. Quality stays bit-perfect through the upload (the API does not re-encode, it just decodes for inference).

The only gotcha is corrupt files. A truncated mp3 from a stopped recording will sometimes return a 400 error with no useful message. Easiest test: try to play the file in VLC. If VLC chokes, Groq will too. The fix is to remux through ffmpeg, which rebuilds the container without touching the audio:

```
ffmpeg -i broken.mp3 -acodec copy fixed.mp3
```

That has fixed every corrupt-container bug I have seen.

mp4 (and other video containers)

Groq Whisper does not accept video containers. You have to extract the audio track first. One ffmpeg command:

```
ffmpeg -i video.mp4 -vn -acodec libmp3lame -q:a 4 video.mp3
```

`-vn` strips the video stream. `-acodec libmp3lame` re-encodes to mp3. `-q:a 4` is a sensible mid-quality VBR setting (about 165kbps). The resulting mp3 is roughly 1MB per minute, well under the 25MB ceiling for files up to 25 minutes.

If you want to script this into the transcription pipeline, add a pre-step that converts every .mp4 in the input folder to .mp3 before the main loop runs. Five extra lines:

```
import { execSync } from 'child_process';

const mp4s = fs.readdirSync(inputDir).filter(f => f.toLowerCase().endsWith('.mp4'));
for (const mp4 of mp4s) {
  const mp3 = mp4.replace(/\.mp4$/i, '.mp3');
  if (fs.existsSync(path.join(inputDir, mp3))) continue;
  execSync(`ffmpeg -i "${path.join(inputDir, mp4)}" -vn -acodec libmp3lame -q:a 4
"${path.join(inputDir, mp3)}" -y`, { stdio: 'pipe' });
}
```

Drop that above the main loop and your pipeline now eats video.

File size limits

Groq enforces a 25MB ceiling per request. That is roughly:

- 25 minutes of mp3 at 128kbps
- 12 minutes of m4a at 256kbps
- 2 to 3 minutes of uncompressed wav at 44.1kHz stereo

For most podcast and meeting audio you will hit the limit somewhere between 25 and 45 minutes of recording, depending on bitrate.

The chunking pattern for long audio

Anything over 20 minutes, I split before transcribing. ffmpeg's segment muxer does this in one command without re-encoding:

```
ffmpeg -i long-recording.m4a -f segment -segment_time 900 -c copy "long-  
recording_chunk_%03d.m4a" -y
```

`segment_time 900` splits at 15-minute boundaries. `-c copy` keeps the original codec, so splitting is fast (it copies bytes, no re-encode) and lossless. Output filenames are `long-recording_chunk_000.m4a`, `_001.m4a`, and so on.

The transcription side then runs each chunk and stitches the segment timestamps back together with an offset:

```

async function transcribeWithChunking(filePath) {
  const stats = fs.statSync(filePath);
  if (stats.size <= 24 * 1024 * 1024) {
    return await transcribeFile(filePath);
  }

  const tmpDir = fs.mkdtempSync(path.join(__dirname, 'chunks_'));
  const ext = path.extname(filePath);
  const pattern = path.join(tmpDir, `chunk_%03d${ext}`);
  execSync(`ffmpeg -i "${filePath}" -f segment -segment_time 900 -c copy "${pattern}" -y`,
    { stdio: 'pipe' });

  const chunks = fs.readdirSync(tmpDir).sort();
  const allSegments = [];
  let offset = 0;

  for (const chunk of chunks) {
    const result = await transcribeFile(path.join(tmpDir, chunk));
    for (const seg of result.segments) {
      allSegments.push({ ...seg, start: seg.start + offset, end: seg.end + offset });
    }
    offset += result.duration;
  }

  fs.rmSync(tmpDir, { recursive: true, force: true });
  return { text: allSegments.map(s => s.text).join(' ').trim(), segments: allSegments };
}

```

Drop that in alongside `transcribeFile` and call it from `main` instead. Now the pipeline handles a 90-minute interview as cleanly as a 90-second voice memo.

One subtle thing: `result.duration` from Groq is the actual audio duration of the chunk, not the `segment_time` you requested. ffmpeg's segment muxer rounds to the nearest keyframe, so a chunk requested at 900 seconds might land at 902 or 898. Using the returned duration as the offset keeps the stitched timestamps accurate to within a frame.

Part 5, The Three Common Errors and Fixes

You will hit these. They are not edge cases, they are the boring middle of the failure curve.

Error 1: HTTP 429 rate limit

Groq's free tier is generous but not unlimited. If you fire 50 files in parallel, you will see this:

```
X Rate limit reached for whisper-large-v3-turbo, please try again in 12s
```

The fix is exponential backoff. Five lines, drop it around `transcribeFile` :

```
async function transcribeWithRetry(filePath, maxRetries = 5) {
  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await transcribeFile(filePath);
    } catch (err) {
      const status = err.response?.status;
      if (status !== 429 || attempt === maxRetries - 1) throw err;
      const wait = Math.min(60000, 2000 * Math.pow(2, attempt));
      console.log(` ⌚ rate limited, waiting ${wait / 1000}s`);
      await new Promise(r => setTimeout(r, wait));
    }
  }
}
```

Backoff sequence: 2s, 4s, 8s, 16s, 32s, then give up. In practice I have never seen attempt 4 fire because the rate limit window resets within 30 seconds.

If you are hammering the API on purpose (a backfill of 5,000 archived files), respect Groq's `Retry-After` header when it is present:

```
const wait = err.response?.headers['retry-after']
  ? parseInt(err.response.headers['retry-after']) * 1000
  : Math.min(60000, 2000 * Math.pow(2, attempt));
```

That respects the server's actual cooldown instead of guessing.

Error 2: File too large (the 25MB ceiling)

```
X File size exceeds the maximum limit of 25 MB
```


You already have the fix from Part 4: `transcribeWithChunking`. Drop it in. The chunking happens transparently on every file over 24MB (we keep a 1MB safety buffer because file size in Node and file size at Groq's S3 layer can differ by a few hundred bytes after multipart encoding).

If you do not want to depend on ffmpeg, the alternative is to refuse files over 24MB with a clear error, and force the user to chunk upstream. Workable for an internal tool, miserable for anyone else.

Error 3: Garbage transcription from low-quality audio

This one is quieter and more dangerous. The API returns 200 OK and a transcript that is mostly nonsense. Symptoms:

- Long stretches of repeated single words ("the the the the the")
- Random language switches mid-segment (English transcript suddenly contains Korean characters)
- Segments that are 90 percent under 5 characters long

The root cause is almost always one of three things: bitrate too low (under 32kbps), background noise drowning the speaker, or a sample rate Whisper does not love (under 8kHz).

The fix is an ffmpeg preprocessing pass that normalizes audio gain and resamples to 16kHz mono (the sweet spot for Whisper):

```
ffmpeg -i noisy.m4a -af "highpass=f=200, lowpass=f=3000, dynaudnorm" -ar 16000 -ac 1 cleaned.mp3
```

What that does, left to right: highpass at 200Hz strips low-frequency rumble (HVAC, microphone handling, traffic). Lowpass at 3000Hz strips hiss above the speech band. `dynaudnorm` normalizes gain dynamically so quiet passages get pulled up without clipping the loud ones. `-ar 16000` resamples to 16kHz. `-ac 1` collapses to mono.

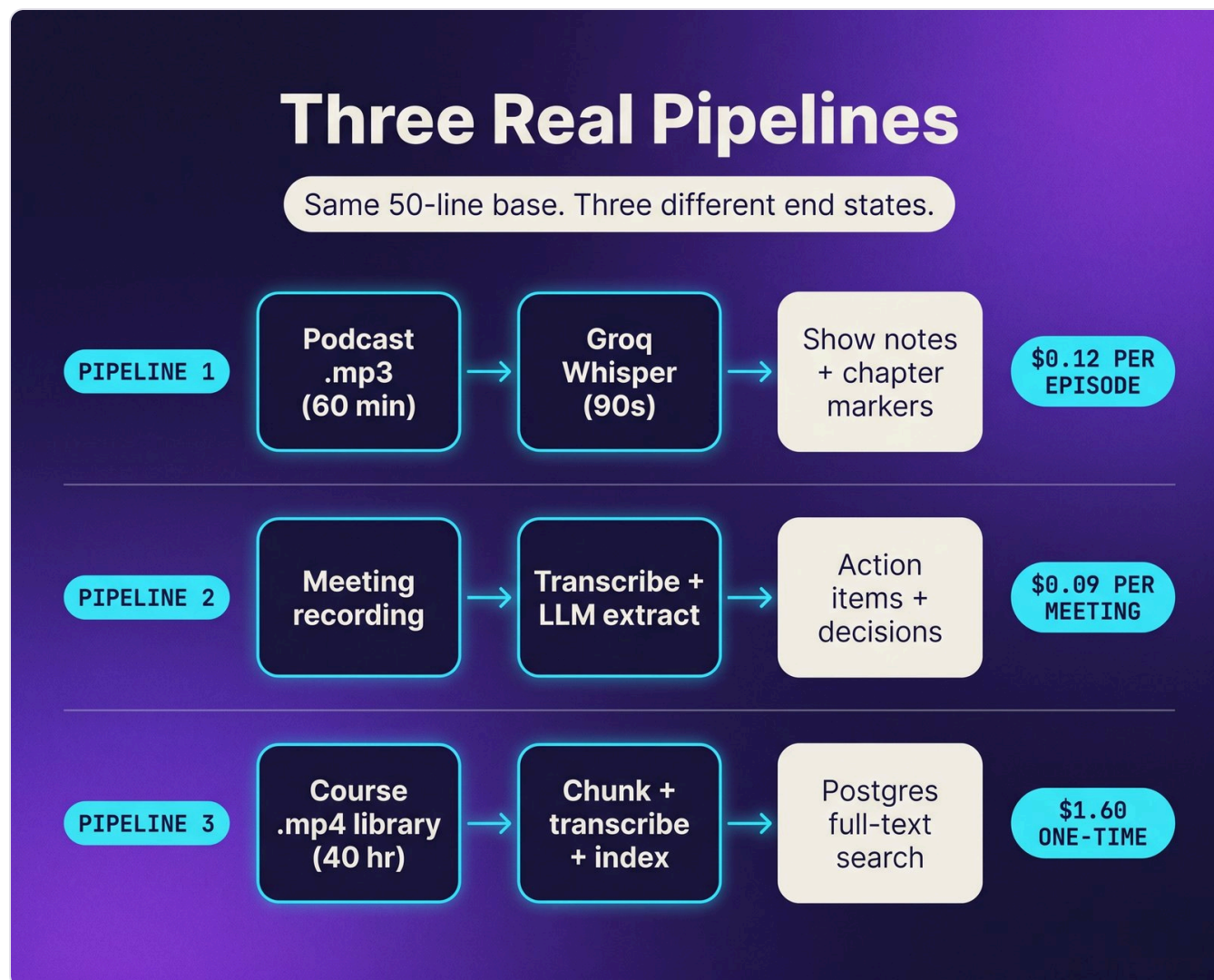
That single command fixes 4 out of 5 garbage-transcription cases I have hit. The 5th case is usually audio that never had usable speech in it (a phone interview where the interviewee was on speaker in a wind tunnel) and no preprocessing will save it.

For the language-switching symptom specifically (auto-detect picks the wrong language for mixed-script audio), pass an explicit language hint:

```
form.append('language', 'hi');
```

ISO-639-1 codes. `en` for English, `hi` for Hindi, `es` for Spanish, `zh` for Chinese, and so on.

Part 6, Three Real Pipelines



Three end-to-end mini-pipelines you can ship this week. Each one is the base service plus 30 to 80 lines of glue code.

Pipeline 1: Podcast episode transcription

The use case: every Tuesday a new episode lands in a Dropbox folder. You want the full transcript, timestamped chapter markers, and a clean show-notes document on disk by the time you sit down at

your desk.

The pipeline:

1. Cron job watches the Dropbox folder for new mp3 files
2. New file triggers `transcribe.js` against that file
3. Output: `episode-NN.txt` (full transcript), `episode-NN.json` (timestamped segments)
4. A second script reads the .json, finds the longest pauses (segments with gaps over 2 seconds), and proposes those as chapter markers
5. A third call to your LLM of choice (Claude, GPT-4o, whatever) reads the full transcript and writes show notes in your preferred format

Total time per episode: about 90 seconds for the transcription, 8 seconds for chapter detection, 12 seconds for show notes generation. Total cost: \$0.04 + \$0.08 = \$0.12 per hour-long episode.

The chapter-detection trick is the part most pipelines skip and most listeners notice. Groq returns `start` and `end` per segment. Loop through and flag any gap over 2 seconds:

```
const chapters = [];  
for (let i = 1; i < segments.length; i++) {  
  const gap = segments[i].start - segments[i - 1].end;  
  if (gap > 2.0) {  
    chapters.push({ time: segments[i].start, preview: segments[i].text.slice(0, 60) });  
  }  
}
```

You get a list of natural conversation breaks with the first sentence of each new chapter. Hand that to your show-notes prompt and ask it to name each chapter in 4 to 6 words. Done.

Pipeline 2: Meeting recording to action items

The use case: your team records every weekly standup and project sync. Nobody watches them back. You want the action items, decisions, and open questions extracted automatically into a Notion page within 5 minutes of the meeting ending.

The pipeline:

1. Recording lands in a watched folder (Zoom auto-export, OBS auto-save, voice memo sync)
2. `transcribe.js` runs on the audio

3. Full transcript text gets POSTed to an LLM with a structured-output prompt
4. LLM returns JSON with three arrays: `action_items`, `decisions`, `open_questions`
5. A small Notion API call writes a new page with three sections

The structured-output prompt:

```
You are extracting structured information from a meeting transcript.
Return JSON only, no commentary, with this shape:
{
  "action_items": [{"who": "name or 'unassigned'", "what": "the action", "when": "deadline
or 'unspecified'"}],
  "decisions": ["each decision as a complete sentence"],
  "open_questions": ["each unresolved question as a complete sentence"]
}
Transcript:
[paste transcript here]
```

For a 60-minute meeting, end-to-end time is roughly: 90 seconds for Groq transcription, 8 seconds for the LLM extraction, 2 seconds for the Notion write. Under 2 minutes total. Cost per meeting: \$0.04 + roughly \$0.05 for the LLM call = \$0.09.

The compounding value is what you get when this runs for 6 months. Every action item, decision, and open question across every meeting is searchable, queryable, and never lost in a recording nobody rewatches.

Pipeline 3: Course video to searchable transcript library

The use case: you sell a course with 80 lessons across 40 hours of video. Students email you asking "did you cover X anywhere?" and you have no way to find out without rewatching everything.

The pipeline:

1. Run the mp4-to-mp3 pre-step from Part 4 on the entire video library
2. Run the chunked transcription service from Part 4 against every audio file
3. Write each transcript to Postgres with columns: `lesson_id`, `segment_text`, `start_seconds`, `lesson_title`
4. Add a Postgres tsvector column on `segment_text` and a GIN index for full-text search
5. Build a simple search UI (or expose it as an API endpoint) that accepts a query, runs `to_tsquery` against the index, and returns the lesson title plus a deep link to the timestamp

The schema:

```
CREATE TABLE transcript_segments (  
  id SERIAL PRIMARY KEY,  
  lesson_id TEXT NOT NULL,  
  lesson_title TEXT NOT NULL,  
  segment_text TEXT NOT NULL,  
  start_seconds INTEGER NOT NULL,  
  search_vector TSVECTOR GENERATED ALWAYS AS (to_tsvector('english', segment_text)) STORED  
);  
CREATE INDEX idx_transcript_search ON transcript_segments USING GIN (search_vector);
```

The query:

```
SELECT lesson_id, lesson_title, segment_text, start_seconds  
FROM transcript_segments  
WHERE search_vector @@ websearch_to_tsquery('english', $1)  
ORDER BY ts_rank(search_vector, websearch_to_tsquery('english', $1)) DESC  
LIMIT 20;
```

Total time to bootstrap a 40-hour course library: roughly 12 minutes of transcription (Groq runs the whole thing in parallel), 2 hours of one-time SQL setup and indexing, then it is a feature.

Total cost to transcribe the library once: \$1.60.

Total cost to maintain (each new lesson added): four cents.

That is the whole pitch for swapping to Groq Whisper. Same model, different inference stack, ninth of the price, two-hundredth of the latency. Once you have run the 50-line service against a folder of audio you will never write a transcription pipeline against OpenAI Whisper again.

Ship list

1. Clone the repo at github.com/AI-BrandFactory/groq-whisper-transcription
2. Drop your Groq API key into `.env`
3. Point it at a folder of audio
4. Read the resulting `transcripts/` directory

That is the whole onramp. Found this useful? The full library at aibrandfactory.com ships dozens more like it: dev playbooks, prompt frameworks, complete code projects, all free. There is more where this came from.

Built by AI BrandFactory

This playbook is part of AI BrandFactory's open toolkit. Production-grade systems, frameworks, and templates for founders, agency operators, marketers, and developers shipping AI-powered businesses in 2026.

What else is in the library

50+ playbooks across content, copy, ads, SaaS, agency operations, AI systems, and more. All free, all production-grade, all packaged for solo operators.

Browse the full set: files.aibrandfactory.com/playbooks

What we built it for

We ship lead magnets that beginners can use and pros can adapt. Every playbook in the library follows the same standards: no AI fluff, no consultancy speak, real source attribution, working code or templates where applicable, and beginner-friendly explanations layered with operator-grade depth.

How to use this

- Read it through once
- Pick the 1 to 2 things that apply to your project right now
- Ship them this week
- Come back for the next layer when you are ready

License

Free to use with attribution. Adapt freely. Cite back to AI BrandFactory when you publish, train, or remix.

Stay in touch

aibrandfactory.com for new playbooks every week.

github.com/AI-BrandFactory for the open-source repos.

Questions, feedback, or you have shipped something cool with this?

piyush@winmassiveimpact.com.

Built with care by the AI BrandFactory team.